ChatGPT

LSTM-Based Architecture for Sequential Quantum Circuit Inputs

Proven Use of LSTMs in Sequential RL Tasks

LSTM networks (Long Short-Term Memory) are a **well-established choice** for RL tasks involving sequential, variable-length observations. In fact, DeepMind's **A3C** (Asynchronous Advantage Actor-Critic) paper showed that adding a 256-unit LSTM layer on top of a feedforward policy significantly improved performance on partially observed games [1]. Similarly, OpenAI's **Five** Dota2 agent relied on a single-layer **4096-unit LSTM** as the core of each bot's policy network, processing an extremely large state input (on the order of 20,000 features) and outputting a distribution over actions [2]. These successes demonstrate that LSTMs can effectively handle long 1D sequences with repeating patterns in an RL setting.

The advantage of using LSTMs is their ability to **capture long-term dependencies** in a sequence via an internal memory state. This makes them natural for encoding your quantum circuit (a sequence of gate instructions) into a fixed-size representation for decision-making. Even in supervised sequence tasks (e.g. language modeling or text-based games), LSTMs have proven robust. For instance, an LSTM-based agent was used to play **text adventure games** by encoding textual state descriptions into a policy network, yielding good results in those sequential decision problems [3] [4]. Overall, these cases give confidence that an LSTM policy/value network can learn the repeating patterns in your instruction sequences.

Recommended LSTM Network Design for the Policy/Value Network

Given the structure of your input (a sequence of instructions, each represented by a 150-dimensional feature vector), a **stacked LSTM architecture** is recommended. Below are specific design choices, backed by practical research and implementations:

- **Stacked LSTM Layers with Residual Connections:** Using more than one LSTM layer (e.g. 2–3 layers deep) can help the network learn hierarchical features from the sequence. To ensure stable training of a deep LSTM, we can employ **residual connections** between layers. In NLP, a *stacked residual LSTM* (where the input to each LSTM layer is added to its output) was shown to train effectively and outperform shallow LSTMs [5]. You can implement this by feeding the output of layer *k* into layer *k+1* and also adding it (possibly after a linear projection) to the output of layer *k+1*. This technique was used in a paraphrase generation model to successfully train LSTMs 5–7 layers deep [5]. In practice, 2 layers might be sufficient for your case, but the residual connection is a useful addition if you go deeper.

- **Hidden State Size:** The LSTM hidden dimension defines the size of the encoded state representation. Prior work suggests using a **few hundred units** is a good starting point. For example, the A3C agents used a 256-dimensional LSTM state [1], and many open-source RL implementations use 128–256 units for LSTM policies. If your problem is very complex with high-

dimensional observations, you might scale this up (OpenAI Five went up to 4096 units for extreme complexity [2]). A reasonable choice would be **256 or 512 units** for the LSTM hidden size, given your 150-dimensional input features and the complexity of quantum circuits. This size has been **"battle-tested"** in Atari and continuous control benchmarks to provide a good balance between capacity and training stability.

- **Input Encoding/Embedding:** Each time-step input to the LSTM is a 150-length vector encoding one gate instruction. You can feed this directly into the LSTM. However, you might consider adding a **learned linear embedding layer** beforehand that projects the 150 features to the LSTM's hidden size (or another convenient size). This would be analogous to word embeddings in NLP (mapping one-hot words to dense vectors). For example, if your LSTM has 256 units, a linear layer from $\mathbb{R}$^150 → $\mathbb{R}$^256 could be applied to each instruction vector before the LSTM. This could help the network scale/normalize the different components (gate type, qubit flags, parameters) more flexibly. Many implementations simply let the first LSTM layer itself do this projection (since the input-to-hidden weights can learn an embedding), so this layer is optional. If the range of the numeric gate parameters varies widely, you may want to **normalize those features** or at least initialize the network in a way to handle unscaled values.

- **Actor-Critic Structure:** For an advantage actor-critic (A2C/A3C-style) network, you will have **two output heads** from the LSTM encoder: one for the policy (action probability distribution π) and one for the value function V. There are two common design patterns here:

- **Shared LSTM encoder:** Use one LSTM stack to encode the input sequence into a final hidden state. Then apply two separate feedforward layers to that state – one outputs the policy logits and another outputs the scalar value. This approach is simpler and has fewer parameters. It was used, for example, in the recurrent PPO reference implementation: a single LSTM processes observations, then its output is passed through one hidden fully-connected layer and split into value and policy streams [6]. In your case, you could take the final LSTM hidden state h_T (after processing the last instruction in the circuit) and feed it into (a) a fully-connected layer to N action logits and (b) a fully-connected layer to 1 value output. If needed, you can include an intermediate FC layer with nonlinearity for each head (e.g. DeepMind's IMPALA agent and Stable-Baselines use a small MLP on top of the LSTM) [7].

- **Separate LSTM for actor and critic:** To avoid any interference between policy and value learning, some implementations give each head its *own* LSTM module. For instance, Stable Baselines3's recurrent PPO defines two LSTMs with identical architecture – one feeds into the policy MLP, the other feeds into the value MLP [8]. This *Actor-Recurrent-Critic (ARC)* design doubles the recurrent parameters but can improve stability if the value function updates were destabilizing the shared representation. Both approaches have been used successfully in practice. If your training is stable, the shared approach is perfectly fine (and more efficient). If you observe training difficulties, the separate LSTM approach (as in SB3 or in some research on LSTM-augmented DDPG/PPO [9] [10]) is a fallback.

- **Activation Functions and Normalization:** Inside the LSTM, the gating uses sigmoids/tanh by design, so you typically don't need to add extra activation there. For the output head layers, **ReLU or LeakyReLU** are common choices (many RL implementations use ReLU in the policy/value MLP). In the example recurrent PPO architecture, all FC layers after the LSTM used ReLU [11], whereas Stable

Baselines3 used Tanh in a small two-layer MLP [7] – either can work, with ReLU being a safe default. You can also consider applying **layer normalization** to the LSTM (there are variants of LSTM that include layer norm on the gates, which can stabilize training on noisy RL data). If using PyTorch, `nn.LSTM` doesn't have built-in layernorm, but you could switch to `nn.LSTMCell` and manually add `nn.LayerNorm` on the cell/hidden states. This is an optional tweak – many baseline implementations did fine *without* layernorm or weight norm on the LSTM. (Weight normalization is more common in conv nets like your TCN [6], but for LSTM a better analogue is layer norm or recurrent dropout.)

- **Sequence Processing Details:** Because each quantum circuit can have a different length of instructions, you'll need to handle variable-length sequences. The LSTM can process them by either **padding** to a max length or by looping over time steps until the sequence ends. A typical RL implementation would pad sequences in a batch to equal length and use a mask (to ignore padded timesteps in loss computations). If you roll out one environment at a time, you can also feed the sequence step-by-step into the LSTM and take the final state. In either case, ensure you **reset the LSTM's hidden state** at the start of each new sequence (episode) so that each circuit is encoded independently (unless you have a reason to carry state across episodes). The open-source truncated BPTT PPO implementation provides examples of how to manage hidden states and sequence resets in an RL training loop [12] [13].

Optional Enhancements: Attention and Hybrid Models

While a standard stacked LSTM should work well, there are a couple of extensions you might consider if you need extra performance:

- **Attention Mechanism on LSTM Outputs:** *Attention* isn't limited to Transformers – one can attach an attention layer on top of an LSTM to help it focus on important parts of the sequence. For example, in NLP classification tasks, adding a **soft attention pooling** over the LSTM's outputs often improved accuracy by letting the model weight which words (timesteps) were most relevant to the prediction [14]. In your context, an attention module could learn to assign higher weight to certain critical gate instructions in the circuit when computing the policy or value. Implementationally, you would take the sequence of LSTM hidden states $(h_1, \ldots, h_T)$ and compute attention weights over them (via a small feedforward attention network, or even a simple dot-product query if you have a fixed context query). Then compute a context vector as the weighted sum of hidden states, and feed that to the policy/value heads. This adds complexity, but *if* you suspect that not all instructions are equally important (perhaps some pattern in the middle of the circuit heavily influences the reward), attention could be beneficial. There are code examples of adding attention to LSTM in open source – e.g., a bi-LSTM+attention for sentiment analysis (with code on GitHub) reported better performance than LSTM alone [14]. Since you prefer to avoid full Transformers, this lightweight attention is a reasonable compromise if needed.

- **Hybrid CNN/TCN + LSTM Architectures:** If pure LSTM or pure TCN struggle to capture the sequence structure alone, you can combine them. A **hybrid TCN-LSTM** model could, for instance, use convolutional layers (TCN) to extract local patterns from the instruction sequence, then an LSTM to capture longer-term dependencies. Some research in time-series forecasting found that such hybrid models outperformed either component alone (one study reported a TCN-LSTM achieved ~33% higher accuracy than a standalone LSTM on a particulate matter forecast task) [15]. For your case,

one imaginable hybrid is to apply a 1-D CNN or TCN over the sequence of instructions to get an initial set of features (capturing any short-range repeating patterns of gates), then feed the sequence of CNN outputs into an LSTM for global order processing. This approach is more complex and computationally heavy; given that you already have a well-performing TCN design, it might be more straightforward to keep the designs separate (pure TCN vs pure LSTM) for your benchmarking. But it's good to know that if a single-model approach underperforms, combining the **temporal convolution** (for pattern extraction) with **LSTM** (for order/memory) is a viable path (and indeed has open-source examples in other domains).

Implementation Resources

Crucially, you requested architectures with available implementations. Here are a few relevant resources that you can consult or draw inspiration from:

- **Stable Baselines3 – Recurrent PPO:** The *sb3-contrib* repository contains `RecurrentPPO` with an `MlpLstmPolicy`. It uses two LSTM layers (actor & critic) of 256 units each by default, and wraps the PPO algorithm to train with truncated BPTT. The architecture (as printed in an issue discussion) shows a Flatten → LSTM(...,256) → two-stream MLP design [16] [8]. This is a robust, industry-tested implementation; you can find the code in the SB3-Contrib GitHub and documentation [7] [8]. Adapting that architecture to libtorch C++ would be straightforward (libtorch has an LSTM module; you'd just need to handle the hidden state logic manually).

- **MarcoMeter's Recurrent PPO (PyTorch)** – This open-source project implements PPO with truncated BPTT and supports LSTM or GRU policies [17] [18]. Their README and code illustrate how to organize sequences, reset hidden states, etc., and the included example uses (for visual input) a CNN encoder followed by an LSTM [6]. For purely vector input, their design would feed the observation directly into the LSTM. This repo can serve as a **reference for training procedure** and verification that the architecture works in practice. It's also easy to modify the network definition there (`model.py`) to add residual connections or attention if you desire.

- **Residual LSTM Seq2Seq (Paraphrase Generation):** The GitHub repo `iamaaditya/neural-paraphrase-generation` provides an implementation of a stacked residual LSTM network for sequence-to-sequence tasks. By running it with the `--use_residual_lstm` flag, it enables skip connections across LSTM layers [5] [19]. This implementation is in TensorFlow, but it concretely demonstrates the training of deeper LSTMs. Since your use-case is not seq-to-seq but rather sequence-to-policy, you wouldn't need the decoder part; however, the encoder (which is essentially multiple LSTMs in stack) is directly applicable. It's a good example if you want to see how residual connections were added and verify that it indeed improved training as reported in their paper.

- **Attention Layer Examples:** There are plenty of code snippets for adding an attention layer on top of an LSTM. One simple reference is the PyTorch forum discussion on *"Attention for sequence classification using LSTM"*, which provides code for an attention mechanism on the LSTM outputs [20]. Additionally, the W&B report by Aman Arora (linked above) contains a clear PyTorch implementation of a Bi-LSTM with attention (and a comparison of performance) [14]. These can guide you in implementing the attention module if you choose to include it.

In summary, **a 1D LSTM-based architecture is a strong candidate** for your problem. A suitable design is to use an LSTM (or stacked LSTMs) to encode the variable-length circuit sequence into a fixed-size hidden state, then use feedforward layers to output the action probabilities and value. This design is conceptually simple and has been proven in both the research literature and real-world RL implementations:

- It handles sequences of varying length inherently (via hidden state carry-over and masking).
- It can capture repeating patterns through its gating mechanisms and long memory.
- It has known successful examples in similar settings (from text-based RL to partially observable physics tasks).

By leveraging implementations like SB3's recurrent policy or the referenced GitHub projects, you can jump-start integrating a **deep LSTM policy network** in your C++ (LibTorch) pipeline. The combination of **residual connections** (to go deep), possibly **attention** (to highlight key instructions), and well-chosen hyperparameters (layers, units as noted above) should give you a robust model to compare against your TCN. Since you're generating random circuits every episode (no fixed dataset), overfitting isn't a concern – so feel free to make the network reasonably large if needed. The primary concern will be training stability and convergence speed, and the design choices above (shared vs separate LSTM, use of layer norm, etc.) are aimed at ensuring the LSTM learns effectively.

References:

- DeepMind A3C paper – used a 256-cell LSTM to improve agent performance on partially observed environments [1].
- OpenAI Five – each agent was a single-layer 4096-unit LSTM network driving a complex policy (proved an LSTM can scale to very large sequential inputs) [2].
- Residual LSTM for Paraphrase Generation – demonstrated that adding residual connections between LSTM layers allows training of deeper (3+ layer) LSTM networks successfully [5].
- Stable Baselines3 Recurrent PPO – open-source implementation with separate LSTM policy and value networks (256 units), followed by feedforward layers for policy/value outputs [7] [8].
- Recurrent PPO (Marco et al. 2023) – uses a shared LSTM core and splits into policy/value heads, with ReLU activations [6]. Good reference for sequence handling in RL.
- Attention on LSTM – e.g. adding a learned attention pool over LSTM outputs improved text classification accuracy in a bi-LSTM model [14]. This concept can be adapted to help the policy network focus on crucial parts of the input sequence.
- Hybrid TCN-LSTM – studies in other domains report that combining convolutional sequence modeling with LSTM memory can outperform either alone on certain structured sequence tasks [15]. This is an avenue to explore if pure LSTM doesn't meet expectations.

---

[1] Asynchronous Methods for Deep Reinforcement Learning
http://proceedings.mlr.press/v48/mniha16.pdf

[2] OpenAI Five - Wikipedia
https://en.wikipedia.org/wiki/OpenAI_Five

[3] Reinforcement Learning for Text Games - Leon Overweel
https://leonoverweel.com/projects/2019/mlp-textworld-lstm-dqn/

4 matthewsparr/Deep-Zork: Using NLP and reinforcement learning to ...
https://github.com/matthewsparr/Deep-Zork

5 19 GitHub - iamaaditya/neural-paraphrase-generation: Neural Paraphrase Generation
https://github.com/iamaaditya/neural-paraphrase-generation

6 11 12 13 17 18 GitHub - MarcoMeter/recurrent-ppo-truncated-bptt: Baseline implementation of recurrent PPO using truncated BPTT
https://github.com/MarcoMeter/recurrent-ppo-truncated-bptt

7 8 16 How to use LSTM ? RecurrentPPO from sb3-contrib · Issue #209 · Stable-Baselines-Team/stable-baselines3-contrib · GitHub
https://github.com/Stable-Baselines-Team/stable-baselines3-contrib/issues/209

9 10 Actor -Recurrent -Critic(ARC) networks architecture. The LSTM network... | Download Scientific Diagram
https://www.researchgate.net/figure/Actor-Recurrent-CriticARC-networks-architecture-The-LSTM-network-simulate-the-state_fig4_339059515

14 Sentiment Classification using Bi-LSTM with Attention | Written-Reports – Weights & Biases
https://wandb.ai/amanarora/Written-Reports/reports/Sentiment-Classification-using-Bi-LSTM-with-Attention--Vmlldzo0NzUxNjA5

15 Deep learning coupled model based on TCN-LSTM for particulate ...
https://www.sciencedirect.com/science/article/abs/pii/S1309104223000570

20 Attention for sequence classification using a LSTM - PyTorch Forums
https://discuss.pytorch.org/t/attention-for-sequence-classification-using-a-lstm/26044